TEMA 2

Introducción a Java

Entorno, sintaxis, tipos, operadores y entrada/salida

Propósito del tema

Comprender los conceptos, aplicarlos en Java y ser capaz de explicar las decisiones tomadas.

PROGRAMACIÓN · 1.º DAM

Qué aprenderás

- Comprender el funcionamiento de JDK, bytecode y JVM.
- Crear, compilar y ejecutar programas de consola en Eclipse.
- Usar variables, constantes, tipos, operadores y conversiones.
- Leer datos con Scanner y producir salidas claras.

Ruta de aprendizaje

Conceptos → ejemplos resueltos → práctica guiada → ejercicios de consolidación.

Índice del tema

1. Plataforma Java
2. Proyecto, paquete, clase y método main
3. Identificadores, estilo y comentarios
4. Tipos primitivos y String
5. Variables, constantes y literales
6. Operadores y precedencia
7. Conversiones de tipos
8. Salida por consola
9. Entrada con Scanner
10. Programa completo: ticket de compra
11. Compilar, ejecutar y depurar

1. Plataforma Java

Java combina un lenguaje, una biblioteca estándar y una plataforma de ejecución. El JDK contiene las herramientas de desarrollo; la JVM ejecuta bytecode; el entorno de ejecución aporta las bibliotecas necesarias.

```
Código fuente (.java)
        | javac
        v
Bytecode (.class)
        | JVM
        v
Programa en ejecución
```

Entorno común de aula

Utiliza la misma versión LTS del JDK, la misma edición de Eclipse y una carpeta de trabajo conocida en todos los equipos.

2. Proyecto, paquete, clase y método main

Eclipse organiza el trabajo en proyectos. Dentro de src se crean paquetes y clases. La ejecución de una aplicación de consola comienza en el método main.

```
package tema2;

public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("Hola, DAM");
    }
}
```

- El nombre del archivo debe coincidir con la clase pública.
- Las llaves delimitan bloques; el punto y coma termina sentencias.
- Java distingue mayúsculas y minúsculas.

3. Identificadores, estilo y comentarios

Los nombres deben comunicar intención. Las variables y métodos utilizan lowerCamelCase; las clases, UpperCamelCase; y las constantes, MAYUSCULAS_CON_GUIONES_BAJOS.

```
int numeroAlumnos = 24;
final double IVA = 0.21;

// Explica una decisión concreta
/* Comentario de varias líneas */
```

Código legible

Prefiere precioFinal a pf. Comenta el porqué de una decisión; evita narrar literalmente lo que ya expresa el código.

4. Tipos primitivos y String

Familia	Tipos	Uso habitual
Enteros	byte, short, int, long	Contadores, cantidades, identificadores numéricos
Reales	float, double	Medidas y cálculos con decimales
Carácter	char	Un único carácter Unicode
Lógico	boolean	true o false
Texto	String	Cadenas de caracteres; no es primitivo

```
int edad = 20;
long poblacion = 48_000_000L;
double precio = 19.95;
char grupo = 'A';
boolean matriculado = true;
String nombre = "Lucía";
```

5. Variables, constantes y literales

Declarar reserva un nombre y un tipo; inicializar proporciona el primer valor; asignar sustituye el valor actual. Una constante se declara con final y no puede reasignarse.

```
int unidades;          // declaración
unidades = 3;          // asignación
int stock = 40;        // declaración e inicialización
final double IVA = 0.21;
```

Inicializa antes de usar

Las variables locales deben recibir un valor antes de leerse. Eclipse y el compilador señalarán el uso incorrecto.

6. Operadores y precedencia

Grupo	Operadores	Ejemplo
Aritméticos	+ - * / %	total = precio * unidades
Relacionales	== != > < >= <=	edad >= 18
Lógicos	&& || !	edad >= 18 && tieneEntrada

Grupo	Operadores	Ejemplo
Asignación	= += -= *= /=	contador += 1
Incremento	++ --	intentos++

La multiplicación, división y resto se calculan antes que la suma y la resta. Utiliza paréntesis para hacer explícita la intención.

```
int a = 2 + 3 * 4;       // 14
int b = (2 + 3) * 4;     // 20
int cociente = 7 / 2;    // 3: división entera
double real = 7.0 / 2;   // 3.5
```

7. Conversiones de tipos

Una conversión ampliadora, como int a double, suele ser implícita. Una conversión reductora requiere casting y puede perder información.

```
int unidades = 5;
double valor = unidades;        // 5.0

double temperatura = 23.9;
int parteEntera = (int) temperatura; // 23
```

Cuidado con el casting

Convertir no redondea: descarta la parte decimal. Para redondear utiliza Math.round cuando corresponda.

8. Salida por consola

```
String nombre = "Ana";
double nota = 8.25;

System.out.print("Alumno: " + nombre);
System.out.println(" · Nota: " + nota);
System.out.printf("Nota formateada: %.2f%n", nota);
```

println añade un salto de línea, print mantiene el cursor en la misma línea y printf permite controlar el formato.

9. Entrada con Scanner

```
import java.util.Scanner;

Scanner teclado = new Scanner(System.in);
System.out.print("Introduce tu edad: " );
int edad = teclado.nextInt();

teclado.nextLine(); // consume el salto pendiente
System.out.print("Nombre completo: " );
String nombre = teclado.nextLine();

teclado.close();
```

Método	Lee
nextInt()	Un entero
nextDouble()	Un número real
next()	Una palabra
nextLine()	Una línea completa

Separador decimal

El comportamiento de nextDouble depende de la configuración regional. En ejercicios iniciales puede ser más predecible leer texto y convertirlo.

10. Programa completo: ticket de compra

```
import java.util.Scanner;

public class Ticket {
    public static void main(String[] args) {
        final double IVA = 0.21;
        Scanner teclado = new Scanner(System.in);

        System.out.print("Precio unitario: " );
        double precio = teclado.nextDouble();
        System.out.print("Unidades: " );
        int unidades = teclado.nextInt();

        double subtotal = precio * unidades;
        double cuotaIva = subtotal * IVA;
        double total = subtotal + cuotaIva;

        System.out.printf("Subtotal: %.2f €%n", subtotal);
        System.out.printf("IVA: %.2f €%n", cuotaIva);
        System.out.printf("TOTAL: %.2f €%n", total);
        teclado.close();
    }
}
```

Práctica guiada · Mejora el ticket

Añade el nombre del producto, calcula el precio medio por unidad y muestra todos los importes con dos decimales.

11. Compilar, ejecutar y depurar

Los errores de compilación impiden crear bytecode. Los errores de ejecución aparecen cuando el programa ya se ha iniciado. Los errores lógicos producen resultados incorrectos sin detener el programa.

- Lee primero el mensaje superior o la primera línea señalada.
- Comprueba nombres, tipos, llaves, paréntesis y puntos y coma.
- Imprime valores intermedios o utiliza el depurador de Eclipse.
- Prueba valores normales, límites y entradas inesperadas.

Consolidación · Conversor de segundos

Lee una cantidad total de segundos y muestra horas, minutos y segundos utilizando división entera y el operador resto.

Síntesis y autoevaluación

- El JDK compila y la JVM ejecuta bytecode.
- Java exige tipos, nombres y estructuras sintácticas precisas.
- Los operadores transforman valores y las conversiones pueden perder información.
- Scanner y System.out permiten construir programas interactivos de consola.

Reto final · Comprueba que dominas el tema

Crea un programa que lea nombre, horas trabajadas y precio por hora. Calcula el salario bruto y muestra un informe formateado. Incluye constantes, nombres expresivos y al menos cinco casos de prueba.

Material base

Tema reorganizado y desarrollado a partir de programación-introducción-a-java.pdf, aportado por el docente.